

Robot Bípedo Interactivo — Documentación técnica del proyecto

Documento de referencia para la exposición. Explica qué hace el robot, cómo está construido, qué tecnologías usa y cómo se comunican todas sus partes.

1. ¿Qué es y para qué sirve?

Un **robot bípedo interactivo para promociones comerciales**. El robot recorre un espacio (modo atracción, con las ruedas rodando) para llamar la atención. Cuando una persona se acerca y muestra la mano, el robot **reacciona con un giro**, se detiene y la invita a **escanear un código QR de descuento**. La persona ingresa su **cédula** con botones físicos y recibe un **comprobante en pantalla**. Cada interacción se **registra en una base de datos** (local y en la nube) para llevar control de clientes, productos y ventas.

Idea de valor: convierte una promoción estática en una experiencia interactiva y automatizada, que además captura datos de los clientes.

2. Arquitectura general

El sistema tiene **cuatro capas de cómputo** que se reparten el trabajo según su capacidad, más el celular como cámara:

```
graph TD
    C[Celular (cámara IP)] -- "WiFi (video MJPEG)" --> R[Raspberry Pi 4 (cerebro local)]
    R -- "WiFi / ZeroMQ" --> P[PC (servidor central)]
    R -- "UART (serie)" --> E[ESP32 (control de tiempo real)]
    E -- "UART" --> V[VESC (dual)]
    V --> M[2 motores hub (ruedas)]
```

(visión pesada + base de datos)

(potencia)

Por qué esta división: - La **PC** tiene la potencia para correr visión por computadora (redes neuronales). - La **Raspberry** maneja la interfaz, los botones y coordina todo, pero es liviana. - El **ESP32** hace el control de tiempo real de los motores (timing preciso con FreeRTOS), algo que un sistema operativo como Linux no garantiza. - La **VESC** es el controlador de potencia especializado de los motores.

3. Flujo completo de una interacción

1. **Reposo / atracción:** las ruedas ruedan continuo a baja velocidad (duty 0.10). La cámara busca manos.
 2. **Persona detectada:** al ver la mano, el robot **gira en sentido contrario** 1 segundo (un giro de pocos grados en la maqueta) y **frena**.
 3. **Escaneo QR:** la pantalla pide el QR. La persona lo muestra a la cámara.
 4. **Oferta:** se lee el QR, se muestra el producto y el descuento.
 5. **Cédula:** la persona ingresa su cédula (10 dígitos) con 4 botones físicos. Se valida con el algoritmo **Módulo 10** (validación oficial de cédula ecuatoriana).
 6. **Factura:** se muestra el comprobante con precio base, descuento y total.
 7. **Registro:** la venta se guarda en **SQLite (local)** y se sube a **Firestore (nube)**.
 8. **Cierre:** aparece "muchas gracias" y el robot **vuelve a rodar** (a reposo).
 9. **Timeout de seguridad:** si pasan 20 segundos esperando el QR sin leer nada, el robot agradece igual y vuelve a rodar, para no quedarse trabado.
-

4. Componentes en detalle

4.1 PC — Servidor central

Corre dos programas independientes:

pc_server.py — **Visión por computadora (puerto 5555)** - Recibe los frames de la cámara desde la Raspberry por **ZeroMQ**. - Detecta **manos** con **MediaPipe** (modelo `hand_landmarker.task` de Google): reconoce palma abierta y pulgar arriba contando dedos y landmarks. - Lee **códigos QR** con **OpenCV** (con una segunda pasada en escala de grises + umbral Otsu para mejorar QRs con poco contraste). - Le devuelve a la Pi banderas de control: `{hand_detected, thumbs_up, qr_data}`.

api_server.py — **Dashboard y registro (puerto 8000)** - Servidor web con **FastAPI**. Expone un **dashboard** en `http://localhost:8000` con clientes, productos, transacciones y estadísticas en vivo. - Endpoint `POST /api/transacciones`: recibe la venta de la Pi (cédula + QR), valida el QR contra la BD, calcula precio/descuento, la registra en SQLite y la **espeja en Firestore**.

database.py — **Base de datos local - SQLite** con el ORM **SQLAlchemy**. Tablas: `clientes`, `productos`, `qr_codes`, `transacciones`. - Migración automática y segura (agrega columnas nuevas sin perder datos).

firebase_client.py — **Nube** - Sube cada transacción a **Firestore** (Firestore). Diseño **resiliente**: si no hay internet o credenciales, el robot sigue funcionando con SQLite y las ventas quedan marcadas como pendientes.

sync_firestore.py — **Reintento** - Resube a la nube las transacciones que quedaron pendientes por falta de internet (idempotente: no duplica).

4.2 Raspberry Pi 4 — Cerebro local

pi_gui_gpio.py — Interfaz y coordinación - GUI en Pygame (1280×720) con máquina de estados: **IDLE → QR_SCAN → SALUDO → CEDULA → FACTURA → EXITO → IDLE**. - Lee **4 botones físicos** por GPIO (incrementar, decrementar, OK, borrar) para ingresar la cédula. - Valida la cédula con el algoritmo **Módulo 10**. - Coordina cámara (vía PC), UART (al ESP32) y el registro de ventas (a la API). - Maneja el **modo atracción**: manda **RODAR** al ESP32 en reposo, el giro contrario al ver la mano, y frena al cerrar.

pi_zmq_client.py — Cámara + comunicación con la PC - Toma frames de la **cámara del celular** (cámara IP por WiFi, app IP Webcam) o de una webcam local, los comprime a JPEG y los manda a la PC por **ZeroMQ**.

pi_uart.py — Comunicación con el ESP32 - Manda comandos ASCII (**"COMANDO\n"**) al ESP32 por **UART serie** y espera el **ACK**.

4.3 ESP32 — Control de movimiento (tiempo real)

- Firmware en **C nativo con ESP-IDF y FreeRTOS** (sin Arduino).
- **Dos tareas** corriendo en paralelo:
- **Receptora**: lee los comandos de la Pi por UART, los parsea y responde **ACK**.
- **Control**: ejecuta el movimiento y mantiene el estado de las ruedas.
- **Comandos que entiende**:
- **RODAR:<duty>** → rodar continuo (modo atracción).
- **AVANZAR_T:<ms>:<duty>** → moverse un tiempo a una velocidad y frenar (duty negativo = sentido contrario). Se usa para el giro al ver la mano.
- **PARAR** → frenar.
- Reenvía el duty a la VESC **cada 50 ms**, porque la VESC frena el motor por seguridad si no recibe comandos por ~1 segundo.
- **Andamiaje de sensores infrarrojos** listo (desactivado): cuando se cableen, el robot podrá frenar solo ante un obstáculo durante el avance.

4.4 VESC — Controlador de potencia

- Controlador **VESC dual** (dos unidades en una placa, un motor por rueda), unidas internamente por **CAN bus**.
- El ESP32 le habla a **una VESC** por UART usando el **protocolo binario oficial de VESC** (paquetes con longitud y **CRC16**), y controla la **segunda rueda** reenviando el comando por CAN (**COMM_FORWARD_CAN**).
- Control por **duty cycle** (porcentaje de potencia).

5. Comunicaciones (cómo se hablan las partes)

Enlace	Tecnología	Qué transporta
Celular → Raspberry	WiFi / HTTP (MJPEG)	Video de la cámara

Enlace	Tecnología	Qué transporta
Raspberry ↔ PC	WiFi / ZeroMQ (puerto 5555)	Frames y banderas de visión
Raspberry → PC (API)	WiFi / HTTP REST (puerto 8000)	Registro de ventas
Raspberry ↔ ESP32	UART serie (115200 baud)	Comandos de movimiento
ESP32 → VESC	UART (protocolo binario VESC)	Duty de los motores
VESC ↔ VESC	CAN bus	Comando a la 2ª rueda
PC → Nube	Internet / Firestore	Copia de las ventas

6. Base de datos (local + nube)

Diseño de escritura doble: - **SQLite (local, en la PC)** es la **fuentes de verdad**: rápida y funciona sin internet. - **Firebase / Firestore (nube)** es una **copia consultable** desde cualquier lado. - Cada venta se guarda en las dos. Si se cae internet, la venta **no se pierde**: queda pendiente en SQLite y se sube después con `sync_firebase.py`.

Cómo se consultan los datos: - Dashboard local (`localhost:8000`) — lee de SQLite. - Consola de Firebase → Firestore — ve las ventas en la nube.

7. Tecnologías utilizadas

- **Python** (PC y Raspberry): visión, servidor web, base de datos, interfaz.
- **MediaPipe** (Google) — detección de manos por IA.
- **OpenCV** — lectura de códigos QR y manejo de cámara.
- **ZeroMQ** — mensajería de baja latencia entre Raspberry y PC.
- **FastAPI + Uvicorn** — API REST y dashboard web.
- **SQLAlchemy + SQLite** — base de datos local.
- **Firebase / Firestore** — base de datos en la nube.
- **Pygame** — interfaz gráfica del robot.
- **C / ESP-IDF / FreeRTOS** — firmware del ESP32 (tiempo real).
- **Protocolo binario VESC + CAN bus** — control de los motores.
- **UART, WiFi, HTTP, CAN** — buses y protocolos de comunicación.

8. Decisiones de diseño clave (para defender el proyecto)

- **La visión corre en la PC, no en la Raspberry:** los modelos de IA necesitan potencia; la Pi sola no da abasto con buena latencia.

- **El timing de los motores lo maneja el ESP32:** un microcontrolador con FreeRTOS da tiempos precisos y deterministas; Linux (la Pi) no lo garantiza. Además, así los futuros sensores IR podrán frenar el robot al instante.
 - **Escritura doble en la base de datos:** combina la velocidad y confiabilidad de lo local con la accesibilidad de la nube, sin perder ventas si se cae internet.
 - **El celular como cámara IP:** mejor calidad que una webcam barata (sobre todo de noche) y sin hardware extra.
 - **Protocolo binario VESC real:** la VESC estándar solo entiende su protocolo binario con CRC; se implementó correctamente para controlar ambas ruedas.
-

9. Trabajo a futuro

- **Sensores infrarrojos:** el código ya está preparado; falta cablear los 2 sensores para detección de obstáculos (el robot frena solo durante el avance).
 - **Más movimientos:** el firmware permite agregar nuevos comandos fácilmente.
 - **Consulta en la nube:** una vista/app propia para revisar las ventas de Firebase (por ahora se usan el dashboard local y la consola de Firebase).
-

10. Resumen de una línea

Un robot promocional que **combina visión por IA, una arquitectura distribuida en cuatro capas, control de motores en tiempo real y una base de datos local + en la nube** para automatizar y registrar promociones comerciales de forma interactiva.